

# Asperitas AI Agent

## MVP 개발 과정 및 성능향상 로드맵

MVP-001부터 현재 V1.1B Deploy Guard까지 구현된 내용과, V1.1C 이후 실제 채팅형 RAG 에이전트·Deep Research·멀티 에이전트 분리·ML/DL 고도화까지의 단계별 개발 계획.

Version: 2026-06-26 / 배포용 내부 문서

Status: V1.0.0-internal baseline passed; V1.1A/B/B-Deploy Guard completed; real RAG answer provider pending.

Critical boundary: 현재 앱은 내부 dogfood/dry-run/로그 수집 도구다. 실제 RAG 답변, production DB, autonomous wet-lab, regulatory/clinical/commercial validation은 후속 단계에서 구현·검증되어야 한다.

### Executive Bottom Line

지금까지의 개발은 “멋진 챗봇”을 바로 만드는 방식이 아니라, 출처·검증·보안·감사·워크플로우를 먼저 깔고 그 위에 실제 답변 provider와 retrieval/citation 성능을 얹는 방식으로 진행되었다. 이 순서는 Asperitas의 목표가 일반 챗봇이 아니라 source-grounded, auditable, compliance-aware AI infrastructure이기 때문에 맞다.

| 구분                 | 현재 상태  | 의미                                               |
|--------------------|--------|--------------------------------------------------|
| V1.0.0-internal    | PASS   | 내부 dry-run baseline 및 release evidence 확보        |
| V1.1A              | Merged | failure log schema/collector 구축                  |
| V1.1B              | Merged | Streamlit internal dogfood UI 구축                 |
| V1.1B Deploy Guard | Merged | access key, secrets, controlled internal link 준비 |
| V1.1C              | Next   | 실제 RAG 답변 provider 연결 시작                         |

# 목차

1. 문서 목적과 진실 경계
2. 현재까지 구축된 시스템 요약
3. MVP-001부터 현재까지 개발 타임라인
4. 검증·릴리즈·배포 상태
5. 현재 Streamlit 앱의 역할과 한계
6. V1.1 성능향상 로드맵
7. V1.2 Deep Research / source acquisition 자동화
8. V1.3 독립 에이전트 분리
9. V2/V3 장기 Biological Intelligence Factory
10. 운영 방식: ChatGPT-Codex-GitHub-Claude Code
11. 배포·보안·컴플라이언스 게이트
12. 참고자료

# 1. 문서 목적과 진실 경계

이 문서는 Asperitas AI Agent의 전체 개발 과정을 “무엇을 했고, 앞으로 무엇을 할지”로 정리한 내부 배포용 로드맵이다. 팀원, 개발자, 리뷰어가 현재 앱을 오해하지 않도록 현재 구현 범위와 미구현 범위를 분리한다.

내부 문서의 핵심 원칙은 다음과 같다: 이 repository는 generic chatbot이 아니라 source-grounded, auditable, compliance-aware AI operating system의 실행 기반이다. 또한 source map, registry, prompt, scaffold가 존재한다고 해서 production DB, legal license review, vector DB, KG, eval suite, wet-lab validation이 완료된 것은 아니다.

## Truth Boundary

| 혼동하면 안 되는 것                  | 실제 의미                                               |
|------------------------------|-----------------------------------------------------|
| source map / source registry | 자료 목록/등록 구조. production ingestion이 아님               |
| dry-run chat wrapper         | 질문 흐름·보안 워크플로우 검증. 실제 답변 생성이 아님                     |
| Streamlit dogfood app        | 내부 테스트 UI. public SaaS 또는 production deployment가 아님 |
| failure log                  | 성능향상을 위한 실패 데이터. 자동 해결 시스템은 아님                      |
| future ML/DL roadmap         | 계획. 검증된 bio-model 성능이 아님                            |

근거: Asperitas README/AGENTS 계열 문서는 source-grounded, auditable, compliance-aware AI infrastructure를 목표로 하며, source map과 production DB/RAG/KG/eval/wet-lab validation을 혼동하지 말라는 truth boundary를 둔다. Build Guidebook도 첫 목표를 “안전 자율 실험실”이 아니라 출처 기반 내부 의사결정 시스템이라고 명시한다.

## 2. 현재까지 구축된 시스템 요약

현재 시스템은 “실제 답변 AI” 직전 단계까지 도달했다. 핵심 골격은 이미 만들어졌고, 지금 링크 앱은 질문·상태·차단·로그 저장을 위한 internal dogfood UI다.

| 레이어            | 구현된 기능                                                     | 현재 의미                        |
|----------------|------------------------------------------------------------|------------------------------|
| Governance     | AOS/PRIME/AGENTS/README 기반 운영 헌법                           | 개발·답변·보안·진실 경계 기준            |
| Source control | source registry, artifact verification, metadata integrity | 자료 수와 chunk 수의 일관성 검증        |
| Workflow       | workflow planner/run wrapper/inspector/acceptance gate     | 질문이 어떤 경로로 처리되는지 제어          |
| Security       | security guard, high-risk boundary                         | 민감/위험 입력과 overclaim 차단 기반    |
| Audit          | audit trace, decision logs                                 | 검증 가능한 실행 기록                 |
| Chat shell     | ask_asperitas_agent.py dry-run wrapper                     | 아직 provider는 없지만 대화 입구 존재    |
| Dogfood UI     | Streamlit internal app + access guard                      | 링크 기반 내부 테스트와 failure log 저장 |

현재 한 줄 상태: V1은 “답변 전 통제·검증 계층”, V1.1A/B는 “실패 데이터 수집 UI”, V1.1C부터 “실제 RAG 답변 에이전트”가 시작된다.

### 3. MVP-001부터 현재까지 개발 타임라인

아래 표는 MVP-001부터 현재까지의 개발 흐름을 기능 기준으로 재구성한 것이다. MVP-015 이후는 PR/검증 기록 기반으로 더 구체적이며, MVP-001~014는 현재 repo 구조와 최종 검증 결과에서 역추적한 foundation phase다.

| MVP     | 핵심 산출물                                  | 의미                                                        |
|---------|-----------------------------------------|-----------------------------------------------------------|
| MVP-001 | Repository / AOS constitution bootstrap | README, AGENTS, active prompt, truth boundary             |
| MVP-002 | Source folder / governance skeleton     | raw source, disclosure level, protected paths             |
| MVP-003 | Deterministic retriever baseline        | default deterministic retriever; vector/reranker deferred |
| MVP-004 | Metadata schema foundation              | source priority, type, disclosure, verification metadata  |
| MVP-005 | Source registry validation              | registry checks, source count integrity                   |
| MVP-006 | Chunk artifact baseline                 | chunk artifacts and verify_artifacts consistency          |
| MVP-007 | CLI harness / local commands            | local scripts for validation and workflow execution       |
| MVP-008 | Citation/evidence contract scaffold     | source-grounding requirements and answer boundaries       |
| MVP-009 | Compliance/security rule skeleton       | biosafety/compliance/human approval gates                 |
| MVP-010 | Decision log and provenance layer       | 09_LOGS/decision_logs convention                          |
| MVP-011 | Eval fixture / smoke scaffolding        | future eval and regression test basis                     |
| MVP-012 | Release readiness tooling               | readiness checks and protected release conditions         |
| MVP-013 | RAG wrapper dry-run path                | question input without real provider                      |
| MVP-014 | Benchmark/source acquisition prep       | external benchmark strategy and source workflow prep      |
| MVP-015 | Benchmark workflow schema/closeout      | benchmark and stage-gate structure                        |

## MVP-016부터 V1.1B까지 상세 개발 이력

| 단계              | 산출물                                 | 의미                                                                     |
|-----------------|-------------------------------------|------------------------------------------------------------------------|
| MVP-016         | Skills Layer                        | skill registry, discovery validation, benchmark workflow preflight     |
| MVP-017         | Eval Layer                          | eval metric schema, report generator, artifact writer, regression gate |
| MVP-018         | Workflow Layer                      | workflow planner, run wrapper, inspector, acceptance gate              |
| MVP-019A        | Audit Trace Layer                   | 실행·판단·결과 추적 기반                                                         |
| MVP-019B        | Workflow Audit Bridge               | workflow와 audit trace 연결                                               |
| MVP-019C        | Security Guard                      | 보안/민감도/위험 응답 차단                                                        |
| MVP-019D        | Chat QA Workflow Wiring             | 질문 경로와 dry-run 연결                                                      |
| MVP-019E        | V1 Release Closeout                 | V1 release evidence 및 scope closeout                                   |
| PR #82          | V1.0.0-rc1 Closeout Packet          | RC 문서/릴리즈 노트 정리                                                        |
| PR #88          | V1 internal dry-run evidence packet | internal dry-run 재현성 확보                                                |
| V1.0.0-internal | Detached main verification PASS     | 558 tests, artifacts, readiness, smoke, dry-run 통과                     |
| PR #89          | V1.1A Failure Log Collector         | JSONL-friendly failure schema/collector/CLI/docs/tests                 |
| PR #90          | V1.1B Streamlit Internal Dogfood UI | 브라우저 UI, dry-run 결과, failure log save                                  |
| PR #91          | V1.1B Deploy Guard                  | access key, secrets policy, requirements-streamlit, internal link 준비   |

## 4. 검증·릴리즈·배포 상태

현재까지의 검증 흐름은 “기능 추가 → focused tests → full pytest → artifact verification → release readiness → dry-run → PR review → merge”로 진행되었다. 이는 성능 개선 단계에서도 그대로 유지해야 한다.

| 상태                 | 결과                                                    | 해석                              |
|--------------------|-------------------------------------------------------|---------------------------------|
| V1.0.0-internal    | PASS                                                  | internal dry-run baseline 확정    |
| V1.1A              | merged / focused 11 passed / full 569 passed reported | failure log collector 구축        |
| V1.1B              | merged / focused 8 passed / full 577 passed reported  | Streamlit dogfood UI 구축         |
| V1.1B Deploy Guard | merged / focused 13 passed / full 582 passed reported | controlled internal link 준비     |
| Streamlit link     | 사용자가 생성                                               | 팀원에게 제한적으로 공유 가능하나 dry-run only |

### 현재 배포의 성격

- 내부 팀원 링크 공유용 dogfood 배포다.
- real RAG answer provider는 아직 구현 전이다.
- 팀원에게 “완성형 AI”가 아니라 “질문/실패 데이터 수집용”이라고 안내해야 한다.
- 비밀번호/access key를 사용하고, secrets는 GitHub에 올리지 않는다.

Streamlit Community Cloud는 workspace에서 Create app을 누르고 app 정보를 입력한 뒤 Deploy하는 흐름이며, dependencies와 secrets 관리가 필요하다. Streamlit 문서는 secrets를 repository에 커밋하지 말고 app settings/secrets로 넣으라고 안내한다.

## 5. 현재 Streamlit 앱의 역할과 한계

| 가능한 것                         | 아직 안 되는 것                                      |
|-------------------------------|------------------------------------------------|
| 질문 입력, dry-run 상태 확인          | ChatGPT처럼 source-grounded 답변 생성                |
| security/workflow/audit 상태 확인 | 내부 문서 citation 제공                              |
| failure log JSONL 저장          | vector DB / hybrid search / reranker           |
| access key로 내부 링크 보호          | Deep Research 자동 수집                            |
| 20~50개 dogfood 질문 수집          | production SaaS / external customer deployment |

### Dogfood 운영법

- 처음에는 2~5명에게만 링크를 준다.
- 총 20~50개 질문을 수집한다.
- 질문은 실제 업무형: 회사소개, DBTL, 합성생물학, 논문 요약, source gap, 전략 질문 위주로 한다.
- 민감정보, API key, 비공개 원문 전체, 규제/실험 민감 세부정보는 입력하지 않는다.
- 실패하면 category/severity/status를 선택해 failure log를 저장한다.

이 로그는 V1.1C provider 연결, V1.1D eval, V1.1E citation/evidence 품질 개선의 입력 데이터가 된다.



## 6. V1.1 성능향상 로드맵

V1.1의 핵심 목표는 “실제로 질문하면 내부 자료를 근거로 답변하고, 그 답변 품질을 평가·개선하는 루프”를 만드는 것이다. 로그 수집과 성능향상은 병행한다.

| 단계    | 구현 기능                              | 언제/왜                                                                           |
|-------|------------------------------------|--------------------------------------------------------------------------------|
| V1.1C | Real RAG Answer Provider           | dry-run 대신 실제 LLM/provider 연결. 근거 없으면 제한 답변. 내부 사용 가능한 채팅 시작점.                 |
| V1.1D | Retrieval/Answer Baseline Eval     | 질문세트, expected answer, faithfulness/relevance/context metrics 구축. 개선 전후 비교 가능. |
| V1.1E | Citation & Evidence Quality        | 답변마다 source/chunk/citation 연결. unsupported claim 차단. 내부 연구팀 사용 가능 수준 시작.       |
| V1.1F | Source Gap Report                  | 자료가 없거나 부족한 질문을 감지하고 “source gap”으로 분류. 수집 우선순위 생성.                            |
| V1.1G | Vector/Hybrid/Reranker Experiments | opt-in 실험으로 검색 성능 개선. 기본값 변경은 eval 통과 후만.                                      |
| V1.1H | Regression Gate Hardening          | 성능 저하 PR merge 차단. 질문 로그 기반 회귀 테스트 자동화.                                        |

OpenAI의 eval guide는 LLM application 성능을 기대 기준에 맞는지 테스트하고, task를 정의하고 test inputs로 실행한 뒤 결과를 분석해 iterate하는 흐름을 강조한다. OpenAI Retrieval 문서는 semantic search와 vector store를 통해 유사 의미 결과를 찾고 모델 응답 합성과 결합하는 구조를 설명한다.

## 7. V1.2: Deep Research / Source Acquisition Automation

V1.2는 “외부 자료를 잘 긁어와서 DB화하는 자동화” 단계다. 단, 저작권/라이선스/공개 범위/PII/내부 기밀 구분이 선행되어야 한다.

| 기능                             | 구현 내용                                                     | 주의                           |
|--------------------------------|-----------------------------------------------------------|------------------------------|
| External source manifest       | 논문, 특허, SEC/IR, 정부/규제, 회사 공식자료 metadata 수집                | 원문 무단 크롤링 금지                 |
| Deep Research collector        | 주제별 최신 자료 탐색, relevance scoring, 중복 제거                    | 공식 API/합법 경로 우선              |
| Source gap to acquisition loop | V1.1F에서 나온 부족 질문을 수집 task로 전환                             | license 상태 확인 전 ingestion 금지 |
| Briefing generator             | CEO/research team용 한국어 briefing 자동 생성                     | citation/source priority 필수  |
| Registry backfill              | source registry에 metadata/license/source type/year/tag 추가 | verify_artifacts/eval 필요     |

OpenAI Deep Research는 주어진 task에 대해 심층적인 정보 수집·합성 흐름을 제공하는 모델/도구 계열로 볼 수 있고, Asperitas에서는 이를 “자동 source acquisition + evidence briefing” 방향으로 통합하는 것이 적합하다.

## 8. V1.3: 독립 에이전트 분리

독립 에이전트 분리는 지금 바로 repo를 여러 개로 쪼개는 것이 아니다. 먼저 single core repository 안에서 agent module을 나누고, 충분히 커진 뒤 별도 service/repo로 분리한다.

| 시점    | 에이전트                          | 역할                                          |
|-------|-------------------------------|---------------------------------------------|
| V1.3A | Research Briefing Agent       | 논문/뉴스/특허/규제 요약 및 relevance 분류               |
| V1.3B | Source Acquisition Agent      | source gap 기반 수집 후보 생성, metadata 생성         |
| V1.3C | Strategy Agent                | 사업/시장/모트/자본전략 판단 지원                         |
| V1.3D | Compliance Agent              | CITES/Nagoya/LMO/biosafety/IP/privacy gate  |
| V1.3E | SynBio Literature Agent       | 합성생물학/DBTL/biofoundry/protein design 질의응답   |
| V1.3F | Grant/RFP Agent               | 정부 과제/RFP/DMP/제안서 구조화                       |
| V1.3G | Experiment Planning Assistant | 고수준 실험 설계 검토. wet-lab 실행은 human approval 이후 |

LangGraph는 long-running, stateful agents를 build/manage/deploy하기 위한 low-level orchestration runtime이며 durable execution, streaming, human-in-the-loop, memory 등을 제공한다고 설명한다. 이런 계층은 V1.3 이후 multi-agent orchestration에 적합하다.

## 9. V2/V3 장기 Biological Intelligence Factory

장기적으로는 RAG chatbot을 넘어서 biological intelligence factory로 확장한다. 그러나 wet-lab, autonomous lab, clinical/regulatory claims는 실제 validation과 human approval이 없으면 절대 완료로 말하지 않는다.

| 버전   | 목표                                  | 핵심 구현                                                                               |
|------|-------------------------------------|-------------------------------------------------------------------------------------|
| V2.0 | Multi-agent Research OS             | planner/executor/reviewer/router/memory, task queues, role-based agents             |
| V2.1 | Knowledge Graph + Entity Linking    | species, gene, protein, pathway, company, regulation 연결                             |
| V2.2 | Benchmark/eval flywheel             | 질문 로그 -> eval set -> regression -> improvement loop                                 |
| V2.3 | AI-native company ops               | daily briefing, Slack/Gmail/GDrive workflows, decision dashboard                    |
| V3.0 | DBTL Planning Layer                 | Design-Build-Test-Learn 계획/기록/분석. human approval gate 필수                            |
| V3.x | Cloud/wet-lab integration candidate | schema-based protocol planning, validation checks, human-in-loop, compliance review |

참고 benchmark: Ginkgo/OpenAI autonomous lab 논문은 LLM과 automated cloud lab을 결합해 CFPS cost/titer optimization을 수행했지만, built-in validation checks, schema constraints, lab automation platform, human intervention boundaries가 포함된 고도화된 실험 환경이다. Asperitas에서 이 단계는 장기 V3 이후 human approval와 실험 인프라가 갖춰진 뒤 가능하다.

## 10. 운영 방식: ChatGPT-Codex-GitHub-Claude Code

| 역할                     | 담당                                                                                         |
|------------------------|--------------------------------------------------------------------------------------------|
| ChatGPT                | architecture lead: roadmap, scope, constraints, prompt, PR review, next-step 판단            |
| Codex                  | implementation engine: code, tests, CLI scripts, docs, PR 생성                               |
| GitHub                 | version control: branch, PR, merge, release evidence, changelog                            |
| Claude Code            | later optimization: refactor, performance, complex code cleanup, large-scale restructuring |
| Human founder/operator | go/no-go, secrets, deployment button, high-risk approval, team feedback collection         |

### 개발 원칙

- 항상 branch -> Codex 구현 -> tests -> PR -> review -> merge.
- retrieval/vector/reranker/default runtime 변경은 eval 없이 금지.
- source 추가는 registry/chunk/artifact 검증 없이는 완료로 말하지 않음.
- production/deployment/clinical/wet-lab/commercial claim 금지.
- 성능향상은 “로그 -> 원인 분류 -> 개선 -> eval -> regression gate”로 진행.

# 11. 배포·보안·컴플라이언스 게이트

현재 배포는 controlled internal dogfood link다. 팀에게 줄 때는 완성형 AI가 아니라 “테스트 페이지”라고 안내해야 한다.

| 게이트                   | 필수 조건                                                                         |
|-----------------------|-------------------------------------------------------------------------------|
| Internal Dogfood Link | access key configured, secrets not committed, dry-run-only warning visible    |
| Real Answer Provider  | provider key in secrets, citation/evidence contract, failure mode tests       |
| Team-wide beta        | V1.1E citation quality + V1.1H regression gate pass                           |
| External pilot        | auth, privacy, compliance, legal review, source licensing, incident plan      |
| Wet-lab planning      | human approval, biosafety/legal review, non-operational high-level boundaries |

팀원 공유 문구 핵심: “dry-run internal dogfood입니다. 민감정보·API key·회사기밀·비공개 원문  
전체·규제/실험 민감 세부정보는 입력하지 마세요. 이상하면 failure log를 저장해주세요.”

## 12. 참고자료

| 구분       | 자료                                                              | 사용 이유                                                                                                |
|----------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Internal | README.md / AGENTS.md / Asperitas AOS docs                      | source-grounded, auditable, compliance-aware AI infrastructure 및 truth boundary                      |
| Internal | Asperitas Codex Master Constitution & DB Integration Guide v1.2 | source map과 production DB/RAG/KG/eval/wet-lab validation 구분                                          |
| Internal | Asperitas Codex AI Agent Build Guidebook v1.0                   | 먼저 source-grounded internal decision system, 완전 자율 실험실은 후순위                                          |
| Web      | OpenAI API - Working with evals                                 | evals로 model outputs를 테스트하고 iterate하는 개발 루프                                                          |
| Web      | OpenAI API - Retrieval                                          | semantic search, vector stores, retrieval+model synthesis                                            |
| Web      | Streamlit Community Cloud docs                                  | Create app / dependencies / secrets management                                                       |
| Web      | LangGraph docs                                                  | stateful long-running agents, durable execution, human-in-loop, memory                               |
| Web      | LlamaIndex docs                                                 | context-augmented LLM apps, data connectors, indexes, query/chat engines, agents, observability/eval |
| Science  | RAGAS paper                                                     | RAG evaluation dimensions: retrieval quality, faithful use of context, generation quality            |
| Science  | Ginkgo/OpenAI autonomous lab paper                              | long-term benchmark for schema-validated autonomous lab integration                                  |
| Science  | de novo protein design review                                   | future ML/DL protein design layer benchmark                                                          |
| Science  | Organoid manufacturing review                                   | future scalable biomanufacturing / reproducibility benchmark                                         |

**End state summary:** V1.1C부터 실제 채팅형 RAG 에이전트가 시작되고, V1.1E 이후 citation/evidence가 붙어 내부 연구팀 사용 수준에 가까워진다. V1.2는 Deep Research/source acquisition, V1.3은 독립 에이전트 분리, V2/V3는 multi-agent biological intelligence factory와 DBTL planning으로 확장한다.